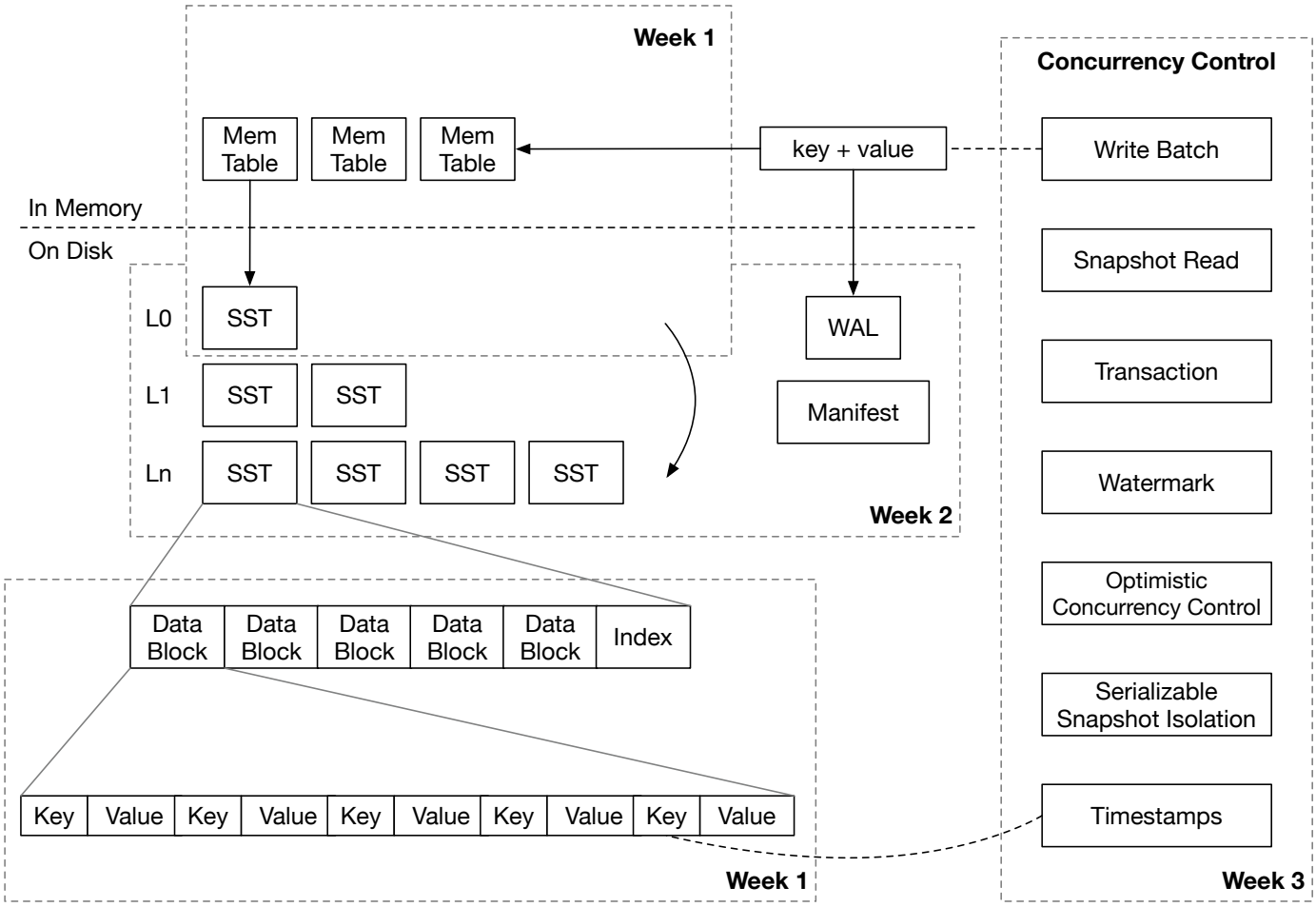
Mini-LSM Course Overview

Course Structure



We have three parts (weeks) for this course. In the first week, we will focus on the storage structure and the storage format of an LSM storage engine. In the second week, we will deeply dive into compactions and implement persistence support for the storage engine. In the third week, we will implement multi-version concurrency control.

- The First Week: Mini-LSM
- The Second Week: Compaction and Persistence
- The Third Week: Multi-Version Concurrency Control

Please look at Environment Setup to set up the environment.

Overview of LSM

An LSM storage engine generally contains three parts:

1. Write-ahead log to persist temporary data for recovery.
2. SSTs on the disk to maintain an LSM-tree structure.
3. Mem-tables in memory for batching small writes.

The storage engine generally provides the following interfaces:

- `Put(key, value)` : store a key-value pair in the LSM tree.
- `Delete(key)` : remove a key and its corresponding value.
- `Get(key)` : get the value corresponding to a key.
- `Scan(range)` : get a range of key-value pairs.

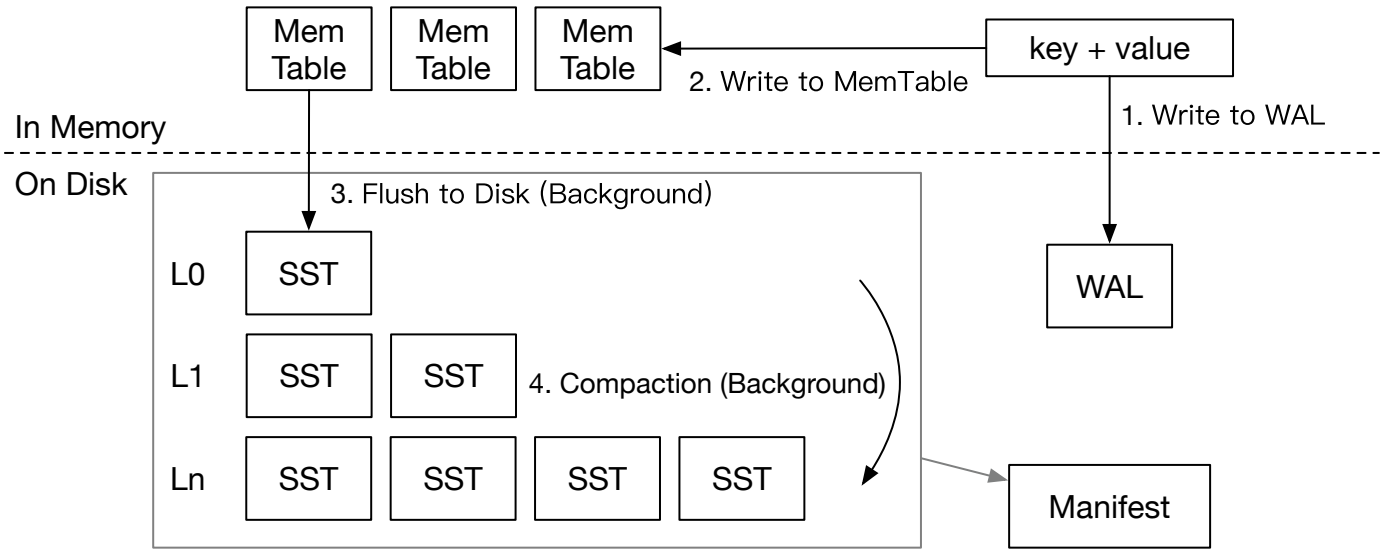
To ensure persistence,

- `Sync()` : ensure all the operations before `sync` are persisted to the disk.

Some engines choose to combine `Put` and `Delete` into a single operation called `WriteBatch`, which accepts a batch of key-value pairs.

In this course, we assume the LSM tree is using a leveled compaction algorithm, which is commonly used in real-world systems.

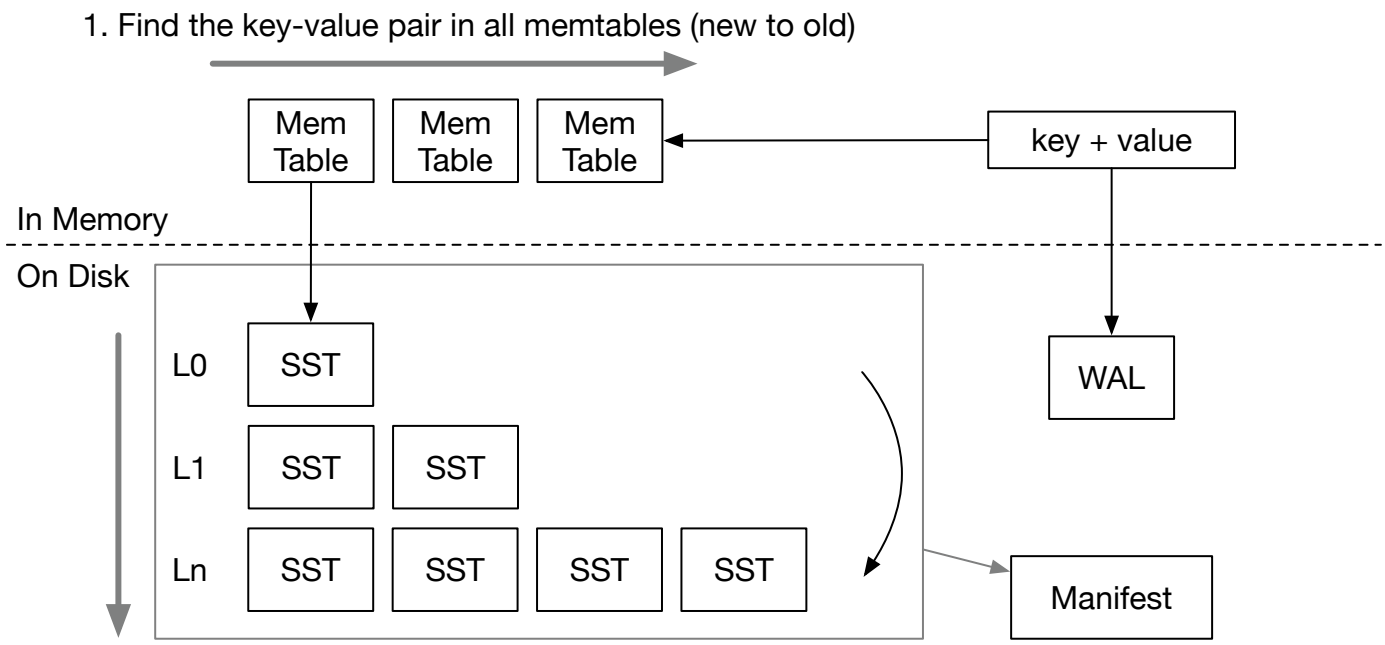
Write Path



The write path of LSM contains four steps:

1. Write the key-value pair to the write-ahead log so that it can be recovered after the storage engine crashes.
2. Write the key-value pair to memtable. After (1) and (2) are completed, we can notify the user that the write operation is completed.
3. (In the background) When a mem-table is full, we will freeze them into immutable mem-tables and flush them to the disk as SST files in the background.
4. (In the background) The engine will compact some files in some levels into lower levels to maintain a good shape for the LSM tree so that the read amplification is low.

Read Path



When we want to read a key,

1. We will first probe all the mem-tables from the latest to the oldest.
2. If the key is not found, we will then search the entire LSM tree containing SSTs to find the data.

There are two types of read: lookup and scan. Lookup finds one key in the LSM tree, while scan iterates all keys within a range in the storage engine. We will cover both of them throughout the course.